

BỘ BÀI TẬP SYSTEMVERILOG

Mạch tổ hợp, mạch dãy và yêu cầu viết testbench trên ModelSim

Học phần	Điện tử số / Thiết kế số
Ngôn ngữ HDL	SystemVerilog (IEEE 1800)
Công cụ mô phỏng	ModelSim / QuestaSim
Cấu trúc	10 bài mạch tổ hợp + 10 bài mạch dãy

Mục tiêu: giúp sinh viên luyện tập mô tả phần cứng, tư duy thiết kế RTL và xây dựng testbench tự kiểm tra trên ModelSim.

Hướng dẫn chung

- Mỗi bài phải nộp tối thiểu 02 file: file RTL .sv và file testbench tb_*.sv.
- Mã RTL phải viết theo SystemVerilog; dùng always_comb cho logic tổ hợp và always_ff cho logic tuần tự khi phù hợp.
- Không dùng #delay trong module RTL tổng hợp. #delay chỉ dùng trong testbench.
- Testbench phải có kiểm tra tự động bằng assert hoặc if/\$error; không chấp nhận chỉ chụp waveform mà không có tiêu chí PASS/FAIL.
- Chạy mô phỏng trên ModelSim/Questasim và lưu waveform cho các tình huống chính.

Danh mục 20 bài

Bài	Nhóm	Tên bài	Mức độ
1	Mạch tổ hợp	Bộ chọn dữ liệu MUX 4:1, dữ liệu 8 bit	Cơ bản
2	Mạch tổ hợp	Decoder 3-to-8 có tín hiệu Enable	Cơ bản
3	Mạch tổ hợp	Priority Encoder 8-to-3 kèm valid	Cơ bản–Trung bình
4	Mạch tổ hợp	Bộ so sánh hai số 8 bit signed/unsigned	Trung bình
5	Mạch tổ hợp	Bộ cộng/trừ 8 bit dùng bù 2	Trung bình
6	Mạch tổ hợp	ALU 8 bit – 8 phép toán	Trung bình
7	Mạch tổ hợp	Bộ cộng một chữ số BCD	Trung bình
8	Mạch tổ hợp	Giải mã Hex sang LED 7 đoạn	Cơ bản–Trung bình
9	Mạch tổ hợp	Barrel Shifter 8 bit hai chiều	Trung bình
10	Mạch tổ hợp	Tạo và kiểm tra parity chẵn cho dữ liệu 16 bit	Trung bình
11	Mạch dây	D Flip-Flop có enable và reset đồng bộ	Cơ bản
12	Mạch dây	Bộ đếm tăng 4 bit có enable và reset bất đồng bộ	Cơ bản
13	Mạch dây	Bộ đếm Up/Down 8 bit có load song song	Trung bình

Bài	Nhóm	Tên bài	Mức độ
14	Mạch dây	Thanh ghi dịch 8 bit SIPO/PISO	Trung bình
15	Mạch dây	Bộ phát PWM 8 bit	Trung bình
16	Mạch dây	FSM Moore phát hiện chuỗi 1011 có chồng lấp	Trung bình–Khá
17	Mạch dây	FSM điều khiển đèn giao thông hai hướng	Khá
18	Mạch dây	Bộ lọc chống dội nút nhấn theo số mẫu liên tiếp	Khá
19	Mạch dây	FIFO đồng bộ sâu 4, dữ liệu 8 bit	Khá
20	Mạch dây	Bộ đếm thời gian BCD 00–59 giây với start/stop	Khá

Bài 1. Bộ chọn dữ liệu MUX 4:1, dữ liệu 8 bit

Mức độ: Cơ bản

Đề bài

Thiết kế bộ MUX 4:1 có bốn ngõ vào dữ liệu 8 bit. Tín hiệu sel chọn một trong bốn ngõ vào đưa ra y.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
d0,d1,d2,d3	input	8 bit	Bốn ngõ vào dữ liệu
sel	input	2 bit	Tín hiệu chọn
y	output	8 bit	Dữ liệu được chọn

Yêu cầu thiết kế

- Viết module mux4_1_8bit bằng SystemVerilog.
- Ưu tiên always_comb và câu lệnh unique case.
- Không sử dụng delay (#) trong RTL.

Yêu cầu testbench trên ModelSim

- Tạo đủ 4 trường hợp sel = 00, 01, 10, 11.
- Thay đổi dữ liệu đầu vào giữa các lần kiểm tra.
- Testbench phải tự so sánh y với giá trị mong đợi và báo \$error khi sai.

Gợi ý: Có thể viết một task check(input logic [1:0] s, input logic [7:0] exp).

Bài 2. Decoder 3-to-8 có tín hiệu Enable

Mức độ: Cơ bản

Đề bài

Thiết kế decoder 3-to-8 active-high. Khi $en=1$, đúng một bit của $y[7:0]$ bằng 1 tương ứng với giá trị $a[2:0]$; khi $en=0$, $y=0$.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
a	input	3 bit	Mã nhị phân vào
en	input	1 bit	Cho phép hoạt động
y	output	8 bit	Ngõ ra one-hot

Yêu cầu thiết kế

- Sử dụng phép dịch hoặc case.
- Đảm bảo đầu ra one-hot khi $en=1$.

Yêu cầu testbench trên ModelSim

- Quét a từ 0 đến 7 với $en=1$.
- Kiểm tra $en=0$ với ít nhất ba giá trị a khác nhau.
- Dùng assertion tức thời hoặc if/\$error.

Gợi ý: Khi $en=1$, biểu thức $8'b1 \ll a$ tạo đúng một bit 1.

Bài 3. Priority Encoder 8-to-3 kèm valid

Mức độ: Cơ bản–Trung bình

Đề bài

Thiết kế bộ mã hóa ưu tiên 8 ngõ vào. Bit có chỉ số lớn hơn có ưu tiên cao hơn. code trả về chỉ số của bit 1 ưu tiên cao nhất; valid=1 nếu din khác 0.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
din	input	8 bit	Ngõ vào yêu cầu
code	output	3 bit	Chỉ số ưu tiên
valid	output	1 bit	Có ít nhất một yêu cầu

Yêu cầu thiết kế

- Ưu tiên din[7] cao nhất, din[0] thấp nhất.
- Không tạo latch.

Yêu cầu testbench trên ModelSim

- Kiểm tra từng one-hot input.
- Kiểm tra ít nhất 5 trường hợp có nhiều bit 1.
- Kiểm tra din=0 làm valid=0.

Gợi ý: Gán mặc định code=0, valid=0 rồi xét if...else if từ bit 7 xuống bit 0.

Bài 4. Bộ so sánh hai số 8 bit signed/unsigned

Mức độ: Trung bình

Đề bài

Thiết kế comparator 8 bit có mode chọn cách diễn giải dữ liệu: mode_signed=0 so sánh unsigned, mode_signed=1 so sánh số bù 2 signed.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
a,b	input	8 bit	Hai toán hạng
mode_signed	input	1 bit	0: unsigned; 1: signed
lt,eq,gt	output	1 bit	Quan hệ nhỏ hơn/bằng/lớn hơn

Yêu cầu thiết kế

- Ba đầu ra phải loại trừ lẫn nhau.
- Dùng \$signed khi so sánh signed.

Yêu cầu testbench trên ModelSim

- Kiểm tra biên 8'h7F, 8'h80, 8'hFF, 8'h00.
- Chứng minh cùng một mẫu bit có thể cho kết quả khác giữa signed và unsigned.

Gợi ý: \$signed(a) ép kiểu vector logic thành giá trị signed.

Bài 5. Bộ cộng/trừ 8 bit dùng bù 2

Mức độ: Trung bình

Đề bài

Thiết kế mạch thực hiện $y=a+b$ khi $sub=0$ và $y=a-b$ khi $sub=1$. Xuất $carry_out$ và $overflow$ cho số signed.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
a,b	input	8 bit	Toán hạng
sub	input	1 bit	0:cộng; 1:trừ
y	output	8 bit	Kết quả
carry_out	output	1 bit	Carry/borrow encoding
overflow	output	1 bit	Tràn số signed

Yêu cầu thiết kế

- Dùng công thức $a + (b \text{ XOR } \{8\{sub\}\}) + sub$.
- Tính overflow đúng cho cả cộng và trừ.

Yêu cầu testbench trên ModelSim

- Kiểm tra cộng thường, trừ thường, carry và overflow.
- Tạo reference model 9 bit trong testbench.

Gợi ý: Overflow cộng: cùng dấu toán hạng nhưng kết quả khác dấu. Overflow trừ: a và b khác dấu nhưng y khác dấu a.

Bài 6. ALU 8 bit – 8 phép toán

Mức độ: Trung bình

Đề bài

Thiết kế ALU 8 bit với opcode 3 bit thực hiện: ADD, SUB, AND, OR, XOR, NOT A, shift left A, shift right logical A. Xuất zero flag.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
a,b	input	8 bit	Toán hạng
op	input	3 bit	Mã phép toán
y	output	8 bit	Kết quả
zero	output	1 bit	1 khi y=0

Yêu cầu thiết kế

- Dùng typedef enum logic [2:0] cho opcode.
- Dùng always_comb và case.

Yêu cầu testbench trên ModelSim

- Kiểm tra tất cả 8 opcode.
- Dùng hàm reference trong testbench để tính kết quả mong đợi.

Gợi ý: enum giúp mã dễ đọc và giảm “magic numbers”.

Bài 7. Bộ cộng một chữ số BCD

Mức độ: Trung bình

Đề bài

Thiết kế mạch cộng hai chữ số BCD a,b (0..9) và carry_in. Kết quả gồm sum_bcd (0..9) và carry_out.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
a,b	input	4 bit	Hai chữ số BCD hợp lệ 0..9
carry_in	input	1 bit	Carry vào
sum_bcd	output	4 bit	Chữ số hàng đơn vị
carry_out	output	1 bit	Carry hàng chục

Yêu cầu thiết kế

- Tính tổng nhị phân 5 bit.
- Nếu tổng >9 thì cộng hiệu chỉnh 6.

Yêu cầu testbench trên ModelSim

- Quét toàn bộ $10 \times 10 \times 2 = 200$ tổ hợp hợp lệ.
- So sánh với phép toán số nguyên trong testbench.

Gợi ý: Đây là bài phù hợp để testbench exhaustive vì không gian đầu vào nhỏ.

Bài 8. Giải mã Hex sang LED 7 đoạn

Mức độ: Cơ bản–Trung bình

Đề bài

Thiết kế bộ giải mã hex 0..F sang LED 7 đoạn common-cathode, thứ tự seg={a,b,c,d,e,f,g}, active-high.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
hex	input	4 bit	Giá trị 0..F
seg	output	7 bit	a..g active-high

Yêu cầu thiết kế

- Mô tả bằng case đủ 16 giá trị.
- Quy ước rõ thứ tự bit để tránh nhầm chân LED.

Yêu cầu testbench trên ModelSim

- Kiểm tra đủ 16 giá trị hex.
- Ít nhất kiểm tra chắc chắn 0,1,2,A,F bằng hằng mong đợi.

Gợi ý: Ví dụ số 0 sáng a,b,c,d,e,f và tắt g: 7'b1111110.

Bài 9. Barrel Shifter 8 bit hai chiều

Mức độ: Trung bình

Đề bài

Thiết kế bộ dịch 8 bit. shamt chọn số bit dịch 0..7; dir=0 dịch trái logic, dir=1 dịch phải logic.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
din	input	8 bit	Dữ liệu vào
shamt	input	3 bit	Số bit dịch
dir	input	1 bit	0:left, 1:right
dout	output	8 bit	Dữ liệu ra

Yêu cầu thiết kế

- Mô tả thuần tổ hợp.
- Không dùng vòng lặp cần thiết nếu toán tử <<, >> đã biểu diễn trực tiếp chức năng.

Yêu cầu testbench trên ModelSim

- Quét shamt 0..7 cho cả hai hướng với vài pattern dữ liệu.
- Dùng random test ít nhất 100 mẫu.

Gợi ý: Testbench có thể dùng \$urandom để tạo din, shamt và dir.

Bài 10. Tạo và kiểm tra parity chẵn cho dữ liệu 16 bit

Mức độ: Trung bình

Đề bài

Thiết kế khối tạo even parity cho data[15:0] và khối kiểm tra nhận data + parity_rx, xuất error=1 khi parity không hợp lệ.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
data	input	16 bit	Dữ liệu
parity_even	output	1 bit	Bit parity chẵn
parity_rx	input	1 bit	Parity nhận được
error	output	1 bit	Báo lỗi parity

Yêu cầu thiết kế

- Dùng reduction XOR.
- Tách hoặc gộp generator/checker trong một module đều được.

Yêu cầu testbench trên ModelSim

- Kiểm tra các pattern 0x0000,0xFFFF,0xAAAA,0x1234.
- Random ít nhất 100 mẫu; sau đó cố tình đảo parity_rx để chắc chắn error=1.

Gợi ý: Even parity bit = ^data; checker có thể dùng (^data) ^ parity_rx.

Bài 11. D Flip-Flop có enable và reset đồng bộ

Mức độ: Cơ bản

Đề bài

Thiết kế D-FF cạnh lên. Reset active-high đồng bộ có ưu tiên cao hơn enable. Khi reset=0 và en=1 thì q nhận d; khi en=0 thì giữ nguyên.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	input	1 bit	Clock cạnh lên
reset	input	1 bit	Reset đồng bộ
en	input	1 bit	Cho phép nạp
d	input	1 bit	Dữ liệu
q	output	1 bit	Trạng thái

Yêu cầu thiết kế

- Dùng always_ff @(posedge clk).
- Không dùng #delay trong RTL.

Yêu cầu testbench trên ModelSim

- Tạo clock chu kỳ 10 ns.
- Kiểm tra reset chỉ có tác dụng tại cạnh lên.
- Kiểm tra giữ q khi en=0.

Gợi ý: Testbench nên kích dữ liệu lệch khỏi cạnh clock để tránh race.

Bài 12. Bộ đếm tăng 4 bit có enable và reset bất đồng bộ

Mức độ: Cơ bản

Đề bài

Thiết kế counter modulo-16 tăng tại cạnh lên khi en=1. Reset active-low bất đồng bộ đưa count về 0 ngay lập tức.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	input	1 bit	Clock
rst_n	input	1 bit	Reset bất đồng bộ active-low
en	input	1 bit	Enable
count	output	4 bit	Giá trị đếm

Yêu cầu thiết kế

- Dùng always_ff @(posedge clk or negedge rst_n).
- Kiểm tra wrap 15 → 0.

Yêu cầu testbench trên ModelSim

- Reset giữa hai cạnh clock và kiểm tra count về 0 ngay.
- Đếm ít nhất 20 xung để thấy wrap.
- Kiểm tra en=0 giữ nguyên count.

Gợi ý: Scoreboard có thể cập nhật expected_count tại mỗi posedge.

Bài 13. Bộ đếm Up/Down 8 bit có load song song

Mức độ: Trung bình

Đề bài

Thiết kế counter 8 bit có reset đồng bộ, load đồng bộ, enable và hướng đếm. Thứ tự ưu tiên: reset > load > enable.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,reset	input	1 bit	Clock, reset đồng bộ
load	input	1 bit	Nạp parallel
load_data	input	8 bit	Giá trị nạp
en	input	1 bit	Cho phép đếm
up	input	1 bit	1:tăng,0:giảm
count	output	8 bit	Trạng thái

Yêu cầu thiết kế

- Thể hiện đúng thứ tự ưu tiên.
- Cho phép tràn modulo 256.

Yêu cầu testbench trên ModelSim

- Kiểm tra reset, load, count up, count down, hold.
- Kiểm tra trường hợp load và en cùng 1: load phải thắng.

Gợi ý: if(reset)... else if(load)... else if(en)...

Bài 14. Thanh ghi dịch 8 bit SIPO/PISO

Mức độ: Trung bình

Đề bài

Thiết kế thanh ghi dịch 8 bit có khả năng nạp song song và dịch phải. Khi load=1 nạp pdata; ngược lại nếu shift_en=1 thì dịch phải, serial_in đi vào MSB. serial_out là LSB hiện tại.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,reset	input	1 bit	Clock/reset đồng bộ
load	input	1 bit	Nạp song song
pdata	input	8 bit	Dữ liệu nạp
shift_en	input	1 bit	Cho phép dịch
serial_in	input	1 bit	Bit nối tiếp vào MSB
q	output	8 bit	Nội dung thanh ghi
serial_out	output	1 bit	LSB hiện tại

Yêu cầu thiết kế

- reset > load > shift_en.
- serial_out gán liên tục từ q[0].

Yêu cầu testbench trên ModelSim

- Nạp 8'b10110010 rồi dịch 8 lần.
- Thu serial_out từng chu kỳ và xác nhận đúng thứ tự LSB-first.

Gợi ý: Dịch phải: q <= {serial_in, q[7:1]}.

Bài 15. Bộ phát PWM 8 bit

Mức độ: Trung bình

Đề bài

Thiết kế PWM sử dụng counter 8 bit chạy tự do. pwm_out=1 khi counter < duty. duty=0 cho 0%, duty=255 xấp xỉ 99.6% nếu counter chạy 0..255.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,rst_n	input	1 bit	Clock/reset bất đồng bộ
duty	input	8 bit	Độ rộng xung
pwm_out	output	1 bit	PWM

Yêu cầu thiết kế

- Counter modulo-256.
- pwm_out là logic tổ hợp so sánh counter với duty.

Yêu cầu testbench trên ModelSim

- Với duty=64, đếm số chu kỳ pwm_out=1 trong một frame 256 clock và phải bằng 64.
- Lặp lại duty=0,128,255.

Gợi ý: Testbench đo duty cycle bằng counter phần mềm.

Bài 16. FSM Moore phát hiện chuỗi 1011 có chồng lấp

Mức độ: Trung bình–Khá

Đề bài

Thiết kế FSM Moore nhận bit serial x tại mỗi cạnh lên và đưa detect=1 khi bốn bit gần nhất tạo chuỗi 1011. Cho phép overlap.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,reset	input	1 bit	Clock/reset đồng bộ
x	input	1 bit	Bit nối tiếp
detect	output	1 bit	Phát hiện chuỗi

Yêu cầu thiết kế

- Dùng typedef enum logic cho trạng thái.
- Tách next-state logic và state register.
- Cho phép overlap, ví dụ stream 1011011 có hai lần detect.

Yêu cầu testbench trên ModelSim

- Kích chuỗi 1011011.
- Đếm số chu kỳ detect=1 và xác nhận bằng 2.
- Quan sát state trên waveform nếu simulator cho phép.

Gợi ý: Các trạng thái biểu diễn tiền tố đã khớp: S0, S1(1), S10(10), S101(101), S1011.

Bài 17. FSM điều khiển đèn giao thông hai hướng

Mức độ: Khá

Đề bài

Thiết kế bộ điều khiển đơn giản hai hướng NS và EW. Mỗi hướng có Green/Yellow/Red. Sử dụng bộ đếm thời gian nội bộ: Green 5 chu kỳ, Yellow 2 chu kỳ. Không bao giờ được phép hai hướng cùng Green.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,reset	input	1 bit	Clock/reset đồng bộ
ns,ew	output	3 bit mỗi hướng	Mã {R,Y,G}

Yêu cầu thiết kế

- FSM gồm NS_G, NS_Y, EW_G, EW_Y.
- Bộ đếm reset khi chuyển trạng thái.
- Tạo đầu ra an toàn: một hướng xanh thì hướng kia đỏ.

Yêu cầu testbench trên ModelSim

- Mô phỏng ít nhất hai chu kỳ hoàn chỉnh của FSM.
- Assertion rằng !(ns[0] && ew[0]) tại mọi cạnh clock.
- Kiểm tra thời lượng mỗi trạng thái.

Gợi ý: Có thể dùng counter 3 bit và ngưỡng 4 cho Green, 1 cho Yellow.

Bài 18. Bộ lọc chống dội nút nhấn theo số mẫu liên tiếp ➔ done

Mức độ: Khá

Đề bài

Thiết kế debouncer đồng bộ: btn_raw được lấy mẫu mỗi clock. Chỉ thay đổi btn_clean khi quan sát 4 mẫu liên tiếp khác trạng thái hiện tại. Giả sử tín hiệu đã qua đồng bộ hóa clock-domain.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,reset	input	1 bit	Clock/reset
btn_raw	input	1 bit	Tín hiệu nút đã đồng bộ
btn_clean	output	1 bit	Tín hiệu đã lọc

Yêu cầu thiết kế

- Dùng counter đếm số mẫu liên tiếp khác btn_clean.
- Khi btn_raw quay lại bằng btn_clean thì counter về 0.

Yêu cầu testbench trên ModelSim

- Tạo các xung nhiễu 1–3 clock và đảm bảo output không đổi.
- Giữ mức mới ít nhất 4 clock để output đổi.

Gợi ý: Counter 2 hoặc 3 bit; khi cnt==3 và mẫu vẫn khác thì cập nhật output.

Bài 19. FIFO đồng bộ sâu 4, dữ liệu 8 bit → done

Mức độ: Khá

Đề bài

Thiết kế FIFO single-clock độ sâu 4. Hỗ trợ push khi `wr_en && !full` và pop khi `rd_en && !empty`. Xuất `full`, `empty` và `rd_data`.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
<code>clk,rst_n</code>	input	1 bit	Clock/reset async
<code>wr_en,rd_en</code>	input	1 bit	Ghi/đọc
<code>wr_data</code>	input	8 bit	Dữ liệu ghi
<code>rd_data</code>	output	8 bit	Dữ liệu đọc
<code>full,empty</code>	output	1 bit	Cờ trạng thái

Yêu cầu thiết kế

- Dùng mảng logic `[7:0] mem[0:3]`, con trỏ đọc/ghi 2 bit và count 3 bit.
- Không ghi khi full, không đọc khi empty.
- Xử lý push và pop đồng thời để count không đổi.

Yêu cầu testbench trên ModelSim

- Ghi 4 byte và kiểm tra full.
- Đọc 4 byte theo đúng FIFO order và kiểm tra empty.
- Thử ghi khi full và đọc khi empty; trạng thái phải không hỏng.

Gợi ý: case (`{push,pop}`) 2'b10: `count+1`; 2'b01: `count-1`; default: giữ.

Bài 20. Bộ đếm thời gian BCD 00–59 giây với start/stop → done

Mức độ: Khá

Đề bài

Giả sử tick_1hz là xung enable một chu kỳ clock mỗi giây. Thiết kế bộ đếm hai chữ số BCD 00..59 có run, reset và clear. Khi run=1 và tick_1hz=1, tăng 1 giây; sau 59 quay về 00.

Giao diện tín hiệu

Tín hiệu	Hướng	Độ rộng	Mô tả
clk,reset	input	1 bit	Clock/reset đồng bộ
tick_1hz	input	1 bit	Enable 1 Hz
run	input	1 bit	Start/stop
clear	input	1 bit	Xóa về 00
tens,ones	output	4 bit mỗi	Hai chữ số BCD

Yêu cầu thiết kế

- Ưu tiên reset > clear > count.
- ones chạy 0..9; tens chạy 0..5.

Yêu cầu testbench trên ModelSim

- Không cần chờ 1 giây thật; phát tick_1hz bằng testbench.
- Tạo 65 tick và xác nhận wrap 59 → 00 rồi tới 05.
- Kiểm tra run=0 giữ nguyên dù tick_1hz=1.

Gợi ý: Mỗi tick có thể phát bằng một chu kỳ clk với tick_1hz=1.

Yêu cầu báo cáo mô phỏng ModelSim

1. Ảnh chụp cửa sổ Transcript thể hiện quá trình compile không có lỗi.
2. Ảnh waveform có tên tín hiệu rõ ràng và đủ thời gian mô phỏng để chứng minh chức năng.
3. Kết quả PASS/FAIL do testbench tự kiểm tra.
4. Giải thích ngắn gọn ít nhất một test case biên hoặc test case quan trọng.